

REST API и интеграции

HTTP и REST по существу, OpenAPI, SOAP, gRPC, GraphQL, брокеры сообщений. Идемпотентность, доставка, ошибки, версионирование и структура постановки на интеграцию.

Редакция от 06.08.2026

Содержание

1. Как системы обмениваются данными
 - 1.1 Синхронный и асинхронный обмен
2. HTTP по существу
 - 2.1 Методы
 - 2.2 Коды ответа
 - 2.3 Заголовки, о которых нужно знать
3. Проектирование REST API
 - 3.1 Ресурсная модель
 - 3.2 Вложенность
 - 3.3 Пагинация, сортировка, фильтрация
 - 3.4 Формат ошибки
 - 3.5 Единый конверт ответа
 - 3.6 Версионирование
4. OpenAPI
 - 4.1 Структура документа
 - 4.2 Пример метода
 - 4.3 Чем спецификация полезна аналитику
5. Аутентификация и авторизация в API
 - 5.1 JWT
 - 5.2 OAuth 2.0
 - 5.3 Модели доступа
6. SOAP и XML-обмен
7. gRPC и GraphQL
8. Асинхронный обмен и брокеры
 - 8.1 Kafka и RabbitMQ
 - 8.2 Понятия, которые спрашивают
 - 8.3 Гарантии доставки
9. Надёжность обмена
 - 9.1 Идемпотентность
 - 9.2 Повторы и таймауты
 - 9.3 Паттерны согласованности
10. Прочие способы обмена
11. Шаблон постановки на интеграцию
12. Чек-лист ревью контракта

1. Как системы обмениваются данными

Прежде чем выбирать протокол, отвечают на четыре вопроса. Ответы определяют весь дальнейший дизайн обмена.

Кто инициирует. Одна система запрашивает данные у другой, или источник сам отправляет изменения. От этого зависит, нужен ли опрос по расписанию, подписка на события или обратный вызов.

Нужен ли ответ немедленно. Если пользователь ждёт результат на экране, обмен синхронный. Если результат нужен «когда-нибудь в ближайшие минуты», асинхронный обмен даёт устойчивость и разгружает обе стороны.

Что происходит при недоступности второй стороны. Ответ на этот вопрос отличает продуманную интеграцию от той, что развалится в первый же день эксплуатации.

Какова цена потери и цена дублирования сообщения. Потерянное уведомление это неприятность, потерянный платёж это инцидент. Продублированное списание денег хуже, чем недоставленное уведомление.

1.1 Синхронный и асинхронный обмен

Признак	Синхронный (HTTP-вызов)	Асинхронный (очередь, событие)
Кто ждёт	Вызывающая сторона блокируется до ответа	Никто не ждёт
Связанность	Жёсткая: приёмник должен быть жив	Слабая: приёмник может быть недоступен
Всплеск нагрузки	Передаётся дальше по цепочке	Гасится очередью
Сложность	Ниже	Выше: нужны идемпотентность, порядок, разбор сбоев
Отладка	Простая: запрос и ответ рядом	Сложная: нужен сквозной идентификатор
Типичное применение	Чтение данных, действие пользователя	Уведомления, интеграция систем, обработка событий

Практическое правило: чтение синхронно, изменение состояния в смежной системе асинхронно, если пользователю не нужен мгновенный результат.

2. HTTP по существу

2.1 Методы

Метод	Назначение	Безопасный	Идемпотентный	Тело запроса
GET	Получить ресурс	да	да	нет
POST	Создать, выполнить действие	нет	нет	да
PUT	Заменить ресурс целиком	нет	да	да
PATCH	Изменить часть ресурса	нет	нет по умолчанию	да
DELETE	Удалить	нет	да	обычно нет
HEAD	Заголовки без тела	да	да	нет
OPTIONS	Доступные методы, предзапрос CORS	да	да	нет

Безопасный означает, что метод не меняет состояние. **Идемпотентный** означает, что повторный одинаковый вызов не меняет результат по сравнению с однократным. Разница важна практически: сеть теряет ответы, клиент повторяет запрос, и от идемпотентности зависит, спишутся деньги один раз или два.

2.2 Коды ответа

Аналитик обязан назначать коды осознанно, потому что от них зависит поведение клиента.

Код	Когда
200	Успех с телом ответа
201	Ресурс создан, в заголовке Location ссылка на него
202	Запрос принят в обработку, результат будет позже
204	Успех без тела (типично для DELETE)
301, 308	Ресурс переехал навсегда
400	Запрос синтаксически некорректен, клиенту нет смысла повторять как есть
401	Не аутентифицирован: нет токена, токен истёк или недействителен
403	Аутентифицирован, но прав недостаточно

Код	Когда
404	Ресурс не найден
405	Метод не поддерживается для этого ресурса
409	Конфликт состояния: попытка создать дубликат, устаревшая версия при оптимистичной блокировке
410	Ресурс существовал и удалён навсегда
415	Неподдерживаемый тип содержимого
422	Синтаксис верен, но данные нарушают бизнес-правила
423	Ресурс заблокирован
429	Слишком много запросов, в заголовке Retry-After время ожидания
500	Ошибка на сервере, клиент не виноват
502, 503, 504	Проблема со смежником, шлюзом или перегрузка; повтор оправдан

Три классических вопроса на собеседовании:

401 против 403. 401 означает «не знаю, кто ты»: токена нет или он недействителен, клиенту следует пройти аутентификацию заново. 403 означает «знаю, кто ты, и тебе нельзя»: повторять с тем же токеном бессмысленно.

404 против 403 при отсутствии прав. Возврат 404 вместо 403 скрывает сам факт существования ресурса. Это осознанный выбор в пользу безопасности: по 403 злоумышленник узнаёт, что объект с таким идентификатором есть.

400 против 422. 400 для случая, когда сервер не смог разобрать запрос (сломанный JSON, отсутствует обязательное поле). 422 для случая, когда запрос понят, но нарушает правила предметной области (дата окончания раньше даты начала).

2.3 Заголовки, о которых нужно знать

`Content-Type` и `Accept` (формат тела и желаемый формат ответа), `Authorization` (учётные данные), `Idempotency-Key` (ключ идемпотентности при повторях), `ETag` и `If-None-Match` (условные запросы и кэширование), `If-Match` (оптимистичная блокировка), `Retry-After` (когда повторить), `X-Request-Id` или `traceparent` (сквозной идентификатор для разбора инцидентов), `Cache-Control`.

Сквозной идентификатор запроса стоит требовать в постановке всегда. Без него разбор инцидента в системе из пяти сервисов превращается в сопоставление времени в журналах вручную.

3. Проектирование REST API

3.1 Ресурсная модель

REST строится вокруг существительных, а не глаголов. Действие выражается методом, объект путём.

Плохо	Хорошо
POST /getApplications	GET /applications
POST /createApplication	POST /applications
POST /deleteApplication?id=5	DELETE /applications/5
GET /applicationsByOrgId/7	GET /applications?organization_id=7
POST /applicationApprove	POST /applications/5/approve

Последняя строка показывает границу подхода. Действие, которое не сводится к созданию, изменению или удалению ресурса, оформляют как подресурс-действие. Это допустимое отступление от чистой ресурсной модели, и спорить о нём на собеседовании не стоит: важно уметь объяснить, почему выбрали такую форму.

Правила именования: существительные во множественном числе, нижний регистр, разделитель в пути дефис, в параметрах и полях тела принятый в проекте стиль (нижнее подчёркивание или camelCase), единообразие важнее выбора.

3.2 Вложенность

`GET /applications/5/attachments` уместно, когда вложение не существует без заявки.

Глубже двух уровней вложенность становится неудобной:

`/organizations/1/applications/5/attachments/9/files/3` лучше заменить на `/files/3` с фильтрами.

3.3 Пагинация, сортировка, фильтрация

Три схемы пагинации:

Схема	Параметры	Плюс	Минус
По смещению	<code>page</code> , <code>limit</code> или <code>offset</code> , <code>limit</code>	Простая, можно прыгнуть на страницу	На больших смещениях медленная, при вставках записи дублируются или пропадают
По курсору	<code>cursor</code> , <code>limit</code>	Стабильна при вставках, быстрая	Нельзя прыгнуть на произвольную страницу

Схема	Параметры	Плюс	Минус
По ключу	<code>after_id</code> , <code>limit</code>	Быстрая, простая	Требует монотонного ключа

Для бесконечной прокрутки в интерфейсе правильнее курсор. Для таблицы с номерами страниц смещение.

Обязательно указывать в постановке: значение `limit` по умолчанию, максимально допустимое значение, что вернётся при превышении, что содержит блок метаданных (общее число записей, признак наличия следующей страницы).

Фильтры: `?status=B+работе&created_from=2026-01-01` . Сортировка: `?sort=-created_at` (минус означает убывание). Выбор полей: `?fields=id,status,created_at` .

3.4 Формат ошибки

Единый формат ошибки экономит недели на стороне потребителей. Минимальный состав: машиночитаемый код, человекочитаемое сообщение, детализация по полям, идентификатор запроса.

```
{
  "success": false,
  "error": {
    "code": "blacklist_exact_match",
    "message": "Заявка не может быть отправлена: объект в чёрном списке",
    "details": [
      { "field": "employees[2].last_name", "reason": "Точное совпадение с записью чёрного списка #418" }
    ],
    "request_id": "01J9F2K7QX8W"
  }
}
```

Правило, которое стоит закрепить в постановке: текст сообщения предназначен человеку и может меняться, код предназначен машине и меняться не должен. Клиент, разбирающий поведение по тексту сообщения, сломается при первой же правке формулировки.

3.5 Единый конверт ответа

Многие команды заворачивают полезную нагрузку в общий конверт с полями успеха, данных, метаданных и ошибки. Подход удобен единообразием разбора. Опасность в половинчатости: если часть методов кладёт объект в конверт, а часть вкладывает его дважды или отдаёт голым, потребитель обязан помнить исключения. Договорённость о конверте либо соблюдается везде, либо не даёт ничего.

3.6 Версионирование

Способ	Пример	Комментарий
В пути	<code>/api/v1/applications</code>	Самый распространённый и наглядный
В заголовке	Accept: <code>application/vnd.company.v2+json</code>	Чище с точки зрения теории, менее удобен в отладке
В параметре	<code>?version=2</code>	Встречается, легко потерять при копировании ссылки

Что считается ломающим изменением: удаление поля, переименование поля, изменение типа, ужесточение обязательности, изменение семантики значения, удаление значения перечисления, изменение кода ответа в успешном сценарии. Такие изменения требуют новой версии.

Что можно делать без новой версии: добавить необязательное поле в ответ, добавить необязательный параметр запроса, добавить новое значение перечисления (если потребители предупреждены, что перечень расширяем), добавить новый метод.

Отсутствие версионирования у API, который заявлен как интеграционный, это архитектурный долг. Первое же изменение контракта ударит по внешнему потребителю, и откатиться будет некуда.

4. OpenAPI

Стандарт описания HTTP-интерфейсов. Умение писать спецификацию руками входит в требования большинства вакансий.

4.1 Структура документа

Разделы верхнего уровня: `openapi` (версия стандарта), `info` (название, версия, описание), `servers` (адреса стенов), `paths` (методы), `components` (переиспользуемые схемы, параметры, ответы, схемы безопасности), `security` (применяемая по умолчанию защита), `tags` (группировка).

4.2 Пример метода

```
paths:
  /applications/{id}/approve:
    post:
      tags: [applications]
      summary: Проголосовать по заявке
      description: >
        Фиксирует решение согласующего. Повторный вызов с тем же решением
        возвращает 200 и не создаёт новую запись в истории.
      parameters:
        - name: id
          in: path
          required: true
          schema: { type: integer, minimum: 1 }
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required: [decision]
              properties:
                decision:
                  type: string
                  enum: [approved, rejected]
                comment:
                  type: string
                  maxLength: 1000
                  description: Обязателен при decision=rejected
            examples:
              отказ:
```

```
        value: { decision: rejected, comment: "Нет доверенности" }
responses:
  '200':
    description: Решение зафиксировано
    content:
      application/json:
        schema: { $ref: '#/components/schemas/ApprovalResult' }
  '403': { description: Пользователь не является согласующим по этой заявке }
  '409': { description: Заявка не в статусе согласования }
  '422': { description: Отказ без комментария }
security:
  - bearerAuth: []
```

4.3 Чем спецификация полезна аналитику

Спецификация это форма постановки, а не документация постфактум. Написанная до разработки, она заставляет ответить на вопросы, которые иначе всплывут в середине спринта: обязательно ли поле, какова максимальная длина, что вернётся при повторном вызове, какие значения допустимы в перечислении.

Дополнительные выгоды: по спецификации генерируются клиенты и заглушки, тестирующий пишет проверки до готовности сервера, фронтенд начинает работу параллельно с бэкендом.

Признак незрелого процесса: спецификация генерируется из кода и не покрывает часть методов. Тогда документ описывает не контракт, а то, что успели разметить, и обещание «описание не расходится с реализацией» становится неполной правдой.

5. Аутентификация и авторизация в API

Аутентификация отвечает на вопрос «кто ты», **авторизация** на вопрос «что тебе можно». Их путают в речи постоянно, и на собеседовании это проверяют.

Механизм	Как работает	Где уместен
API-ключ	Постоянный секрет в заголовке	Простые межсерверные интеграции
Basic	Логин и пароль в заголовке, кодировка base64	Только внутри доверенного контура, поверх TLS
Bearer-токен (JWT)	Подписанный токен с полезной нагрузкой	Веб-приложения, мобильные клиенты
OAuth 2.0	Выдача токена доступа через авторизационный сервер	Доступ от имени пользователя, сторонние приложения
OpenID Connect	Надстройка над OAuth 2.0 для аутентификации	Единый вход
Взаимный TLS	Проверка клиентского сертификата	Банковские и государственные интеграции
Подпись запроса	HMAC от тела и времени	Платёжные шлюзы, вебхуки

5.1 JWT

Три части через точку: заголовок, полезная нагрузка, подпись. Полезная нагрузка не зашифрована, а только закодирована, поэтому класть в неё секреты нельзя. Стандартные поля: `sub` (субъект), `exp` (срок действия), `iat` (время выпуска), `iss` (издатель), `aud` (аудитория).

Практические вопросы, которые аналитик обязан закрыть в постановке: срок жизни токена доступа, механизм обновления через refresh-токен, что происходит при отзыве прав до истечения срока (токен остаётся действительным, и без дополнительной проверки права снимутся только после истечения), где хранится токен на клиенте, что происходит при блокировке учётной записи в середине сессии.

5.2 OAuth 2.0

Основные потоки:

- **Authorization Code с PKCE.** Пользователь входит на стороне провайдера, приложение получает код и меняет его на токен. Стандарт для веб- и мобильных приложений.
- **Client Credentials.** Межсерверный обмен без участия пользователя.

- **Refresh Token.** Обновление токена доступа без повторного входа.

Устаревшие потоки Implicit и Resource Owner Password Credentials в новых системах не применяют.

5.3 Модели доступа

RBAC привязывает права к ролям, роли к пользователям. Простая и понятная модель, покрывает большинство корпоративных систем.

ABAC принимает решение по атрибутам субъекта, объекта и контекста: «руководитель подразделения видит заявки своего подразделения в рабочие часы». Гибче и сложнее.

ACL задаёт права на конкретный объект для конкретного субъекта.

В постановке недостаточно назвать модель. Нужно ответить: видит ли роль объект целиком или частично, различаются ли права на чтение и изменение, наследуются ли права по иерархии подразделений, кто имеет право выдавать права, что происходит с доступом при переводе сотрудника.

6. SOAP и XML-обмен

Живее, чем кажется: банки, телеком, государственные информационные системы и корпоративные шины продолжают работать по SOAP. Аналитик обязан уметь читать WSDL и XSD, даже если сам не проектирует такие интерфейсы.

Структура сообщения: конверт (Envelope), необязательный заголовок (Header) для служебных данных вроде подписи и маршрутизации, тело (Body) с полезной нагрузкой либо с элементом Fault при ошибке.

WSDL описывает интерфейс: типы, сообщения, операции, привязки, адрес службы. **XSD** описывает схему данных с точными типами, обязательностью и допустимыми значениями.

Признак	SOAP	REST
Формат	Только XML	Обычно JSON
Контракт	Строгий, WSDL и XSD	OpenAPI, менее строгий
Транспорт	HTTP, очереди, SMTP	HTTP
Ошибки	Элемент Fault	Коды состояния HTTP
Стандарты безопасности	WS-Security, подпись и шифрование части сообщения	TLS, OAuth
Транзакции	WS-AtomicTransaction	Нет стандарта
Объём	Больше	Меньше

Ответ на вопрос «когда SOAP оправдан»: когда обязателен строгий контракт, требуется подпись отдельных элементов сообщения, нужны стандарты корпоративного уровня или так требует смежная система.

7. gRPC и GraphQL

gRPC. Двоичный протокол поверх HTTP/2, описание контракта в файле protobuf, кодогенерация клиента и сервера. Плюсы: скорость, строгий контракт, потоковая передача в обе стороны. Минусы: тяжелее отлаживать, ограниченная поддержка в браузере. Применяют для обмена между внутренними сервисами.

GraphQL. Одна точка входа, клиент сам указывает нужные поля запросом. Плюсы: нет избыточных и недостающих данных, удобно для разнородных клиентов. Минусы: сложное кэширование, риск тяжёлых запросов, сложнее ограничивать права на уровне поля. Применяют, когда клиентов много и им нужны разные срезы одних данных.

8. Асинхронный обмен и брокеры

8.1 Kafka и RabbitMQ

Признак	RabbitMQ	Kafka
Модель	Брокер очередей с маршрутизацией	Распределённый журнал событий
Судьба сообщения	Удаляется после подтверждения	Хранится заданный срок, читается повторно
Порядок	В пределах очереди	В пределах раздела
Повторное чтение	Нет	Да, сдвигом смещения
Масштабирование потребителей	Конкурирующие потребители	Группы потребителей по разделам
Типичное применение	Задачи, команды, маршрутизация	Поток событий, интеграция, аналитика

8.2 Понятия, которые спрашивают

Топик и раздел. Топик это именованный поток. Раздел это единица параллелизма и порядка: порядок гарантирован внутри раздела, но не между разделами. Ключ сообщения определяет раздел, поэтому события одной сущности с одинаковым ключом сохраняют порядок.

Группа потребителей. Несколько экземпляров сервиса делят разделы между собой, каждое сообщение обрабатывается группой один раз.

Очередь недоставленных сообщений. Куда уходит сообщение, которое не удалось обработать после заданного числа попыток. Обязательный элемент постановки: без неё ошибочное сообщение либо теряется, либо блокирует очередь навсегда.

Подтверждение. Потребитель сообщает брокеру об успешной обработке. Момент подтверждения определяет гарантию: подтверждение до обработки даёт риск потери, после обработки даёт риск дубля.

8.3 Гарантии доставки

Гарантия	Смысл	Цена
Не более одного раза	Сообщение может потеряться, дублей нет	Потеря данных
Не менее одного раза	Потерь нет, возможны дубли	Обязательна идемпотентность приёмника

Гарантия	Смысл	Цена
Ровно один раз	Ни потерь, ни дублей	Достижимо в узких условиях, дорого

На практике выбирают доставку не менее одного раза и делают обработку идемпотентной. Формулировка «нам нужно ровно один раз» на собеседовании это приглашение к разговору о том, почему такая гарантия сквозь несколько систем практически недостижима и чем её заменяют.

9. Надёжность обмена

9.1 Идемпотентность

Свойство операции давать один и тот же результат при повторном выполнении. Ключевое понятие интеграций, и его спрашивают почти на каждом собеседовании.

Как достигается:

- **Ключ идемпотентности.** Клиент генерирует уникальный ключ на операцию и передаёт его в заголовке. Сервер запоминает результат по ключу и при повторе возвращает сохранённый ответ, не выполняя действие заново. Срок хранения ключей указывается в постановке.
- **Естественный ключ.** Номер документа, идентификатор события. Повтор ловится по уникальному ограничению в базе.
- **Проверка состояния.** Операция выполняется, только если объект в подходящем состоянии. Повторное «принять в работу» для заявки, уже находящейся в работе, возвращает текущее состояние вместо ошибки.

Формулировка требования: «повторный вызов метода с тем же ключом идемпотентности в течение 24 часов возвращает результат первого вызова с кодом 200 и не создаёт новых записей».

9.2 Повторы и таймауты

Три числа, которые обязаны быть в постановке любой интеграции: таймаут соединения, таймаут ответа, число повторов и задержка между ними.

Повтор допустим только для идемпотентных операций и только на кодах, где он имеет смысл: сетевая ошибка, 429, 502, 503, 504. Повторять 400 и 422 бессмысленно, ответ не изменится.

Задержка растёт экспоненциально с добавлением случайной составляющей, иначе после восстановления смежника все клиенты ударят по нему одновременно.

Размыкатель цепи. После серии отказов вызовы к смежнику прекращаются на время, чтобы не тратить ресурсы и дать ему восстановиться. Три состояния: замкнут (работаем), разомкнут (не пытаемся), полуразомкнут (пробуем один запрос).

9.3 Паттерны согласованности

Сага. Длинная операция разбивается на локальные транзакции с компенсирующими действиями. Если четвёртый шаг не удался, выполняются компенсации для трёх предыдущих. Применяется там, где распределённая транзакция невозможна.

Транзакционный исходящий ящик. Сообщение записывается в таблицу той же транзакцией, что и изменение данных, а отдельный процесс отправляет его в брокер. Решает классическую проблему: данные сохранились, а событие не отправилось, или наоборот.

Разделение команд и запросов. Модель для записи отделена от модели для чтения. Даёт масштабирование чтения ценой согласованности с задержкой.

Шлюз API и бэкенд для фронтенда. Единая точка входа для внешних потребителей и отдельный слой сборки данных под конкретный клиент.

10. Прочие способы обмена

Вебхук. Обратный вызов: система-источник сама вызывает HTTP-метод получателя при событии. Дёшево и просто. Обязательные пункты постановки: подпись запроса для проверки подлинности, повторы при ошибке получателя, идемпотентность на стороне получателя, что делать при многократном отказе, как получатель отличает повтор от нового события.

Файловый обмен. Выгрузка файла на общий ресурс по расписанию. Живёт в интеграциях с бухгалтерскими и государственными системами. Пункты постановки: формат и кодировка, именование файлов, признак завершённости записи (обычно файл-маркер или переименование после полной записи), поведение при частичной загрузке, хранение обработанных файлов, разбор ошибочных строк.

Опрос по расписанию. Получатель сам периодически спрашивает источник об изменениях. Прост и устойчив, ценой задержки и лишней нагрузки. Уточняется период опроса, признак изменения (метка времени, версия), поведение при пропущенном интервале.

Реплика базы или витрина. Прямой доступ к данным другой системы. Быстро на старте, тяжело в сопровождении: жёсткая связь по внутренней структуре, которая меняется без предупреждения.

11. Шаблон постановки на интеграцию

Проверенная структура. Пустой пункт означает, что вопрос не задан, а не что он неважен.

1. **Назначение.** Какую задачу решает обмен, чей это сценарий.
2. **Участники.** Системы, их роли (источник, приёмник), владельцы, контакты.
3. **Триггер.** Что запускает обмен: действие пользователя, событие, расписание.
4. **Направление и режим.** Одностороннее или двустороннее, синхронное или асинхронное.
5. **Транспорт и протокол.** HTTP, очередь, файл, адреса станций.
6. **Контракт.** Метод, путь, параметры, схема тела запроса и ответа, все коды ответа, примеры.
7. **Маппинг полей.** Таблица соответствия полей источника и приёмника с правилами преобразования и значениями по умолчанию.
8. **Справочники.** Как сопоставляются коды справочников двух систем, что делать при неизвестном значении.
9. **Аутентификация.** Механизм, где хранятся секреты, срок действия, порядок замены.
10. **Ошибки.** Перечень ошибок, реакция на каждую, что видит пользователь, что попадает в журнал.
11. **Идемпотентность.** Ключ, срок хранения, поведение при повторе.
12. **Устойчивость.** Таймауты, повторы, задержки, размыкатель цепи, очередь недоставленных.
13. **Объём и нагрузка.** Средний и пиковый поток, размер сообщения, накопленный объём.
14. **Нефункциональные требования.** Время ответа, доступность, срок доставки события.
15. **Наблюдаемость.** Сквозной идентификатор, что логируется, какие метрики, при каком условии поднимается тревога.
16. **Порядок ввода.** Стенды, тестовые данные, порядок первичной синхронизации накопленных данных.
17. **Открытые вопросы.** Явный список с ответственными.

Пункт про первичную синхронизацию забывают чаще прочих. Интеграция запускается не в пустоту: в обеих системах уже есть данные, и порядок их первичного сведения обычно оказывается отдельным проектом.

12. Чек-лист ревью контракта

Прогонять по чужой или своей спецификации:

- Каждое поле имеет тип, признак обязательности и ограничение длины или диапазона.
- Перечисления заданы явно, указано поведение при неизвестном значении.
- Формат даты и времени задан с часовым поясом.
- Денежные суммы не в числах с плавающей точкой, указана валюта.
- Указан код ответа для каждого исхода, включая отказы по правам и по бизнес-правилам.
- Формат ошибки единый, машиночитаемый код отделён от текста.
- Есть пример запроса и ответа для успеха и хотя бы для одной ошибки.
- Определена пагинация: значение по умолчанию, максимум, поведение при превышении.
- Указано, что происходит при повторном вызове.
- Указаны таймауты и политика повторов.
- Указано, какие поля содержат персональные данные и как они защищены.
- Определён порядок изменения контракта и способ версионирования.
- Указан сквозной идентификатор запроса.
- Спецификация покрывает все методы, которые есть в реализации, а не часть.